SOURCE CODE

auth.py:

```
"""
Authentication module with SQLite database
"""
import sqlite3
from werkzeug.security import generate_password_hash, check_password_hash
from flask_login import UserMixin
import os


DATABASE = 'users.db'


class User(UserMixin):
    def __init__(self, id, username, email):
        self.id = id
        self.username = username
        self.email = email


def init_db():
    """Initialize the database"""
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL
        )
```

```
    ''')
    conn.commit()
    conn.close()

def create_user(username, email, password):
    """Create a new user"""
    try:
        conn = sqlite3.connect(DATABASE)
        cursor = conn.cursor()
        hashed_password = generate_password_hash(password)
        cursor.execute(
            'INSERT INTO users (username, email, password) VALUES (?, ?, ?)',
            (username, email, hashed_password)
        )
        conn.commit()
        conn.close()
        return True, "User created successfully"
    except sqlite3.IntegrityError:
        return False, "Username or email already exists"
    except Exception as e:
        return False, str(e)

def verify_user(username, password):
    """Verify user credentials"""
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute('SELECT id, username, email, password FROM users WHERE
username = ?', (username,))
    user_data = cursor.fetchone()
```

```
    conn.close()

    if user_data and check_password_hash(user_data[3], password):
        return User(user_data[0], user_data[1], user_data[2])
    return None

def get_user_by_id(user_id):
    """Get user by ID"""
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute('SELECT id, username, email FROM users WHERE id = ?',
(user_id,))
    user_data = cursor.fetchone()
    conn.close()

    if user_data:
        return User(user_data[0], user_data[1], user_data[2])
    return None
```

app.py:

```
#!/usr/bin/env python3
"""
Lung Cancer Classification Web App
Upload images and get predictions with confidence scores
"""

from flask import Flask, render_template, request, jsonify, redirect, url_for, flash
from flask_login import LoginManager, login_user, logout_user, login_required,
current_user
import torch
```

```
import torch.nn as nn
import torch.nn.functional as F
from torchvision import transforms
from PIL import Image, ImageDraw
import io
import base64
import os
import numpy as np
import cv2

def is_lung_like(image):
    img = np.array(image)

    # Check if image is too colorful (CT scans are usually grayscale)
    if img.ndim == 3:
        color_diff = np.std(img[:, :, 0] - img[:, :, 1])
        if color_diff > 30:
            return False

    # Check brightness
    if img.mean() > 200:
        return False

    return True


from run_lung_cancer_model import CSPDarkNet53

from auth import init_db, create_user, verify_user, get_user_by_id
```

```
app = Flask(__name__)
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024
app.config['SECRET_KEY'] = 'your-secret-key-change-in-production'



login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'

@login_manager.user_loader
def load_user(user_id):
    return get_user_by_id(int(user_id))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")



MODEL_PATH = 'cspdarknet53_best.pth'
CLASS_NAMES = ['Benign cases', 'Malignant cases', 'Normal cases']

model = None

def load_model():
    """Load the trained model"""
    global model
    if model is None:
        model = CSPDarkNet53(num_classes=len(CLASS_NAMES)).to(device)
        checkpoint = torch.load(MODEL_PATH, map_location=device,
weights_only=False)
```

```
        if isinstance(checkpoint, dict) and 'model_state_dict' in checkpoint:
            model.load_state_dict(checkpoint['model_state_dict'])
        else:
            model.load_state_dict(checkpoint)
        model.eval()
        print(f" Model loaded from {MODEL_PATH}")
    return model


transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])

def generate_gradcam(model, input_tensor, target_layer):
    """Generate Grad-CAM heatmap"""
    gradients = []
    activations = []

    def backward_hook(module, grad_input, grad_output):
        gradients.append(grad_output[0])

    def forward_hook(module, input, output):
        activations.append(output)


    handle_forward = target_layer.register_forward_hook(forward_hook)
```

```
handle_backward = target_layer.register_full_backward_hook(backward_hook)


output = model(input_tensor)
pred_class = output.argmax(dim=1)


model.zero_grad()
output[0, pred_class].backward()


handle_forward.remove()
handle_backward.remove()


gradients_val = gradients[0].cpu().data.numpy()[0]
activations_val = activations[0].cpu().data.numpy()[0]

weights = np.mean(gradients_val, axis=(1, 2))
cam = np.zeros(activations_val.shape[1:], dtype=np.float32)

for i, w in enumerate(weights):
    cam += w * activations_val[i]

cam = np.maximum(cam, 0)
cam = cv2.resize(cam, (224, 224))
cam = cam - np.min(cam)
cam = cam / (np.max(cam) + 1e-8)
```

```
    return cam

def detect_bounding_boxes(heatmap, threshold=0.7):
    """Detect bounding boxes from heatmap with improved accuracy"""

    heatmap_smooth = cv2.GaussianBlur(heatmap, (5, 5), 0)

    binary = (heatmap_smooth > threshold).astype(np.uint8) * 255

    kernel = np.ones((5, 5), np.uint8)
    binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
    binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    if not contours:
        return []

    box_scores = []
    for contour in contours:
        area = cv2.contourArea(contour)
        if area > 500:
            x, y, w, h = cv2.boundingRect(contour)
```

```
            mask = np.zeros_like(heatmap)
            cv2.drawContours(mask, [contour], -1, 1, -1)
            avg_intensity = np.sum(heatmap * mask) / area


            score = area * avg_intensity
            box_scores.append((score, (x, y, w, h)))

    if not box_scores:
        return []


    box_scores.sort(reverse=True)
    top_box = box_scores[0][1]


    x, y, w, h = top_box
    padding = 10
    x = max(0, x - padding)
    y = max(0, y - padding)
    w = min(224 - x, w + 2 * padding)
    h = min(224 - y, h + 2 * padding)

    return [(x, y, w, h)]

def draw_boxes_on_image(image, boxes, predicted_class):
    """Draw bounding boxes on image"""
    if not boxes:
```

```
        return image

    draw = ImageDraw.Draw(image)


    if 'Malignant' in predicted_class:
        box_color = 'red'
        label = 'Cancer Detected'
    elif 'Benign' in predicted_class:
        box_color = 'yellow'
        label = 'Benign Lesion'
    else:
        return image

    for (x, y, w, h) in boxes:

        for i in range(4):
            draw.rectangle([x-i, y-i, x+w+i, y+h+i], outline=box_color)



        label_bbox = draw.textbbox((x, y-20), label)
        draw.rectangle(label_bbox, fill=box_color)
        draw.text((x, y-20), label, fill='black')

    return image

def predict_image(image):
    """Make prediction on uploaded image with bounding boxes"""
    model = load_model()
```

```
original_image = image.convert('RGB')
input_tensor = transform(original_image).unsqueeze(0).to(device)


with torch.no_grad():
    outputs = model(input_tensor)
    probs = torch.softmax(outputs, dim=1)
    confidence, pred_class = torch.max(probs, 1)

confidence_value = confidence.item()

THRESHOLD = 0.7

if confidence_value < THRESHOLD:
    return {
        "predicted_class": "Unknown / Not a lung CT image",
        "confidence": confidence_value * 100,
        "all_probabilities": {},
        "annotated_image": None
    }

all_probs = probs[0].cpu().numpy()
predicted_class = CLASS_NAMES[pred_class.item()]


annotated_image = None
if 'Malignant' in predicted_class or 'Benign' in predicted_class:
```

```
        target_layer = model.stage5.concat_conv.conv


        heatmap = generate_gradcam(model, input_tensor, target_layer)


        boxes = detect_bounding_boxes(heatmap, threshold=0.75)


        annotated_image = original_image.copy()
        annotated_image = annotated_image.resize((224, 224))
        annotated_image = draw_boxes_on_image(annotated_image, boxes,
predicted_class)


    return {
        'predicted_class': predicted_class,
        'confidence': confidence.item() * 100,
        'all_probabilities': {
            CLASS_NAMES[i]: float(all_probs[i] * 100)
            for i in range(len(CLASS_NAMES))
        },
        'annotated_image': annotated_image
    }


@app.route('/')
@login_required
def index():
    """Render main page"""
```

```
    return render_template('index.html', username=current_user.username)


@app.route('/signup', methods=['GET', 'POST'])
def signup():
    """Handle user registration"""
    if current_user.is_authenticated:
        return redirect(url_for('index'))

    if request.method == 'POST':
        username = request.form.get('username')
        email = request.form.get('email')
        password = request.form.get('password')
        confirm_password = request.form.get('confirm_password')

        if password != confirm_password:
            return render_template('signup.html', error='Passwords do not match')

        success, message = create_user(username, email, password)
        if success:
            return redirect(url_for('login', success='Account created successfully! Please
sign in.'))
        else:
            return render_template('signup.html', error=message)

    return render_template('signup.html')


@app.route('/login', methods=['GET', 'POST'])
def login():
    """Handle user login"""
```

```
    if current_user.is_authenticated:
        return redirect(url_for('index'))

    success_msg = request.args.get('success')

    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        user = verify_user(username, password)
        if user:
            login_user(user)
            return redirect(url_for('index'))
        else:
            return render_template('login.html', error='Invalid username or password')

    return render_template('login.html', success=success_msg)

@app.route('/logout')
@login_required
def logout():
    """Handle user logout"""
    logout_user()
    return redirect(url_for('login'))

@app.route('/predict', methods=['POST'])
@login_required
def predict():
    """Handle image upload and prediction"""
```

```
if 'file' not in request.files:
    return jsonify({'error': 'No file uploaded'}), 400

file = request.files['file']

if file.filename == '':
    return jsonify({'error': 'No file selected'}), 400

if not file.filename.lower().endswith(('.png', '.jpg', '.jpeg')):
    return jsonify({'error': 'Invalid file type. Please upload PNG or JPG'}), 400

try:

    image_bytes = file.read()
    image = Image.open(io.BytesIO(image_bytes))

    # Validate image type
    if not is_lung_like(image):
        return jsonify({
            "error": "Uploaded image does not appear to be a lung CT scan"
        }), 400


    result = predict_image(image)


    image_base64 = base64.b64encode(image_bytes).decode('utf-8')
    result['image'] = f"{image_base64}"
```

```
        if result['annotated_image']:
            buffered = io.BytesIO()
            result['annotated_image'].save(buffered, format="JPEG")
            annotated_base64 = base64.b64encode(buffered.getvalue()).decode('utf-8')
            result['annotated_image'] = f"{annotated_base64}"

        return jsonify(result)

    except Exception as e:
        return jsonify({'error': f'Error processing image: {str(e)}'}), 500

@app.route('/health')
def health():
    """Health check endpoint"""
    return jsonify({'status': 'healthy', 'model_loaded': model is not None})

if __name__ == '__main__':

    init_db()


    if not os.path.exists(MODEL_PATH):
        print(f" Error: Model file '{MODEL_PATH}' not found!")
        exit(1)


    load_model()
```

```
    print("\n" + "=" * 60)
    print(" Lung Cancer Classification Web App")
    print("=" * 60)
    print(f" Model loaded successfully")
    print(f" Classes: {CLASS_NAMES}")
    print(f" Device: {device}")
    print("\n Starting server...")
    print(" Open http://localhost:5000 in your browser")
    print("=" * 60 + "\n")

    app.run(debug=True, host='0.0.0.0', port=5000)
```

analyze_model.py:

```
#!/usr/bin/env python3
"""
Comprehensive Model Analysis Script
Evaluates accuracy, precision, recall, F1-score, and confusion matrix
"""

import os
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
from tqdm import tqdm


from run_lung_cancer_model import CSPDarkNet53

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}\n")



def load_model(model_path, num_classes=3):
    """Load the trained model"""
    model = CSPDarkNet53(num_classes=num_classes).to(device)
    checkpoint = torch.load(model_path, map_location=device)

    if isinstance(checkpoint, dict) and 'model_state_dict' in checkpoint:
        model.load_state_dict(checkpoint['model_state_dict'])
    else:
        model.load_state_dict(checkpoint)
    model.eval()
    return model

def load_dataset(data_dir, batch_size=16):
    """Load the dataset for evaluation"""
    eval_transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
    ])
```

```
    dataset = datasets.ImageFolder(data_dir, transform=eval_transform)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=False)

    return loader, dataset

# ============================================================
# Evaluation Functions
# ============================================================

def evaluate_model(model, data_loader):
    """Evaluate model and return predictions and labels"""
    all_preds = []
    all_labels = []
    all_probs = []

    with torch.no_grad():
        for images, labels in tqdm(data_loader, desc="Evaluating"):
            images = images.to(device)
            outputs = model(images)
            probs = torch.softmax(outputs, dim=1)
            _, preds = torch.max(outputs, 1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())
            all_probs.extend(probs.cpu().numpy())

    return np.array(all_preds), np.array(all_labels), np.array(all_probs)
```

```
def calculate_metrics(y_true, y_pred, class_names):
    """Calculate comprehensive metrics"""
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average='weighted', zero_division=0)
    recall = recall_score(y_true, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)



    precision_per_class = precision_score(y_true, y_pred, average=None,
zero_division=0)
    recall_per_class = recall_score(y_true, y_pred, average=None, zero_division=0)
    f1_per_class = f1_score(y_true, y_pred, average=None, zero_division=0)


    return {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'precision_per_class': precision_per_class,
        'recall_per_class': recall_per_class,
        'f1_per_class': f1_per_class
    }


def plot_confusion_matrix(y_true, y_pred, class_names,
save_path='confusion_matrix.png'):
    """Plot and save confusion matrix"""
    cm = confusion_matrix(y_true, y_pred)


    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=class_names, yticklabels=class_names,
            cbar_kws={'label': 'Count'})
    plt.title('Confusion Matrix', fontsize=16, fontweight='bold')
    plt.ylabel('True Label', fontsize=12)
    plt.xlabel('Predicted Label', fontsize=12)
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    print(f" Confusion matrix saved to {save_path}")

    return cm

def plot_class_distribution(dataset, save_path='class_distribution.png'):
    """Plot class distribution"""
    class_counts = {}
    for _, label in dataset.samples:
        class_name = dataset.classes[label]
        class_counts[class_name] = class_counts.get(class_name, 0) + 1

    plt.figure(figsize=(10, 6))
    bars = plt.bar(class_counts.keys(), class_counts.values(), color=['#3498db',
'#e74c3c', '#2ecc71'])
    plt.title('Class Distribution in Dataset', fontsize=16, fontweight='bold')
    plt.xlabel('Class', fontsize=12)
    plt.ylabel('Number of Samples', fontsize=12)
    plt.xticks(rotation=45, ha='right')


    for bar in bars:
        height = bar.get_height()
```

```
        plt.text(bar.get_x() + bar.get_width()/2., height,
                 f'{int(height)}', ha='center', va='bottom', fontsize=10)

    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    print(f" Class distribution saved to {save_path}")

    return class_counts

def plot_per_class_metrics(metrics, class_names, save_path='per_class_metrics.png'):
    """Plot per-class metrics"""
    x = np.arange(len(class_names))
    width = 0.25

    fig, ax = plt.subplots(figsize=(12, 6))
    bars1 = ax.bar(x - width, metrics['precision_per_class'], width, label='Precision',
color='#3498db')
    bars2 = ax.bar(x, metrics['recall_per_class'], width, label='Recall', color='#e74c3c')
    bars3 = ax.bar(x + width, metrics['f1_per_class'], width, label='F1-Score',
color='#2ecc71')

    ax.set_xlabel('Class', fontsize=12)
    ax.set_ylabel('Score', fontsize=12)
    ax.set_title('Per-Class Performance Metrics', fontsize=16, fontweight='bold')
    ax.set_xticks(x)
    ax.set_xticklabels(class_names, rotation=45, ha='right')
    ax.legend()
    ax.set_ylim([0, 1.1])
    ax.grid(axis='y', alpha=0.3)
```

```
    for bars in [bars1, bars2, bars3]:
        for bar in bars:
            height = bar.get_height()
            ax.text(bar.get_x() + bar.get_width()/2., height,
                    f'{height:.2f}', ha='center', va='bottom', fontsize=8)

    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    print(f"✅ Per-class metrics saved to {save_path}")



if __name__ == "__main__":
    print("=" * 70)
    print("🔬 COMPREHENSIVE MODEL ANALYSIS")
    print("=" * 70)


    model_path = 'cspdarknet53_best.pth'
    if not os.path.exists(model_path):
        print(f" Error: Model file '{model_path}' not found!")
        print("   Trying alternative path 'best_lung_cancer_model.pth'...")
        model_path = 'best_lung_cancer_model.pth'
        if not os.path.exists(model_path):
            print(f" Error: Model file not found!")
            exit(1)
```

```
data_dir = 'dataset'
if not os.path.exists(data_dir):
    print(f" Error: Dataset directory '{data_dir}' not found!")
    exit(1)


print("\n Loading model and dataset...")
data_loader, dataset = load_dataset(data_dir)
model = load_model(model_path, num_classes=len(dataset.classes))

print(f" Model loaded from {model_path}")
print(f" Dataset loaded: {len(dataset)} images")
print(f" Classes: {dataset.classes}\n")


print(" 🔄 Evaluating model...")
y_pred, y_true, y_probs = evaluate_model(model, data_loader)


print("\n Calculating metrics...")
metrics = calculate_metrics(y_true, y_pred, dataset.classes)


print("\n" + "=" * 70)
print(" OVERALL PERFORMANCE METRICS")
print("=" * 70)
print(f" Accuracy:  {metrics['accuracy']*100:.2f}%")
print(f" Precision: {metrics['precision']*100:.2f}%")
print(f" Recall:    {metrics['recall']*100:.2f}%")
```

```
print(f" F1-Score:  {metrics['f1_score']*100:.2f}%")



print("\n" + "=" * 70)
print(" PER-CLASS PERFORMANCE METRICS")
print("=" * 70)
for i, class_name in enumerate(dataset.classes):
    print(f"\n{class_name}:")
    print(f"  Precision: {metrics['precision_per_class'][i]*100:.2f}%")
    print(f"  Recall:    {metrics['recall_per_class'][i]*100:.2f}%")
    print(f"  F1-Score:  {metrics['f1_per_class'][i]*100:.2f}%")



print("\n" + "=" * 70)
print(" DETAILED CLASSIFICATION REPORT")
print("=" * 70)
print(classification_report(y_true, y_pred, target_names=dataset.classes))



print("\n Generating visualizations...")
cm = plot_confusion_matrix(y_true, y_pred, dataset.classes)
class_counts = plot_class_distribution(dataset)
plot_per_class_metrics(metrics, dataset.classes)

# Save detailed report
print("\n💾 Saving detailed report...")
with open('model_analysis_report.txt', 'w') as f:
    f.write("=" * 70 + "\n")
```

```
        f.write("LUNG CANCER CLASSIFICATION MODEL - ANALYSIS
REPORT\n")
        f.write("=" * 70 + "\n\n")

        f.write("OVERALL PERFORMANCE METRICS\n")
        f.write("-" * 70 + "\n")
        f.write(f"Accuracy:  {metrics['accuracy']*100:.2f}%\n")
        f.write(f"Precision: {metrics['precision']*100:.2f}%\n")
        f.write(f"Recall:    {metrics['recall']*100:.2f}%\n")
        f.write(f"F1-Score:  {metrics['f1_score']*100:.2f}%\n\n")

        f.write("PER-CLASS PERFORMANCE METRICS\n")
        f.write("-" * 70 + "\n")
        for i, class_name in enumerate(dataset.classes):
            f.write(f"\n{class_name}:\n")
            f.write(f"  Precision: {metrics['precision_per_class'][i]*100:.2f}%\n")
            f.write(f"  Recall:    {metrics['recall_per_class'][i]*100:.2f}%\n")
            f.write(f"  F1-Score:  {metrics['f1_per_class'][i]*100:.2f}%\n")

        f.write("\n" + "=" * 70 + "\n")
        f.write("CLASSIFICATION REPORT\n")
        f.write("=" * 70 + "\n")
        f.write(classification_report(y_true, y_pred, target_names=dataset.classes))

        f.write("\n" + "=" * 70 + "\n")
        f.write("CONFUSION MATRIX\n")
        f.write("=" * 70 + "\n")
        f.write(str(cm) + "\n")
```

```
        f.write("\n" + "=" * 70 + "\n")
        f.write("CLASS DISTRIBUTION\n")
        f.write("=" * 70 + "\n")
        for class_name, count in class_counts.items():
            f.write(f"{class_name}: {count} samples\n")

    print("✅ Detailed report saved to 'model_analysis_report.txt'")

    print("\n" + "=" * 70)
    print("✅ ANALYSIS COMPLETE!")
    print("=" * 70)
    print("\nGenerated files:")
    print("  📄 model_analysis_report.txt")
    print("  📊 confusion_matrix.png")
    print("  📊 class_distribution.png")
    print("  📊 per_class_metrics.png")
```

run_lung_cancer_model.py:

```
#!/usr/bin/env python3
"""
Lung Cancer Classification Model - CSPDarkNet
Adapted from Colab notebook to run locally
"""

import os
import numpy as np
import torch
import torch.nn as nn
```

```
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torch.optim.lr_scheduler import StepLR
from tqdm import tqdm
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import warnings
warnings.filterwarnings("ignore")



device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")




class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=1):
        super().__init__()
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding, bias=False)
        self.bn = nn.BatchNorm2d(out_channels)
        self.act = nn.LeakyReLU(0.1)

    def forward(self, x):
        return self.act(self.bn(self.conv(x)))
```

```
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = ConvBlock(channels, channels // 2, kernel_size=1, padding=0)
        self.conv2 = ConvBlock(channels // 2, channels, kernel_size=3)

    def forward(self, x):
        return x + self.conv2(self.conv1(x))


class CSPBlock(nn.Module):
    def __init__(self, in_channels, out_channels, num_blocks):
        super().__init__()
        self.downsample = ConvBlock(in_channels, out_channels, kernel_size=3,
stride=2)
        self.split_conv0 = ConvBlock(out_channels, out_channels // 2, kernel_size=1,
padding=0)
        self.split_conv1 = ConvBlock(out_channels, out_channels // 2, kernel_size=1,
padding=0)
        self.blocks = nn.Sequential(*[ResidualBlock(out_channels // 2) for _ in
range(num_blocks)])
        self.concat_conv = ConvBlock(out_channels, out_channels, kernel_size=1,
padding=0)

    def forward(self, x):
        x = self.downsample(x)
        x1 = self.blocks(self.split_conv0(x))
        x2 = self.split_conv1(x)
        x = torch.cat((x1, x2), dim=1)
        return self.concat_conv(x)
```

```
class CSPDarkNet53(nn.Module):
    def __init__(self, num_classes=3):
        super().__init__()
        self.conv1 = ConvBlock(3, 32, 3)
        self.stage1 = CSPBlock(32, 64, num_blocks=1)
        self.stage2 = CSPBlock(64, 128, num_blocks=2)
        self.stage3 = CSPBlock(128, 256, num_blocks=8)
        self.stage4 = CSPBlock(256, 512, num_blocks=8)
        self.stage5 = CSPBlock(512, 1024, num_blocks=4)
        self.global_pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(1024, num_classes)

    def forward(self, x):
        x = self.conv1(x)
        x = self.stage1(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.stage4(x)
        x = self.stage5(x)
        x = self.global_pool(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)

        return x



def load_data(data_dir):
```

```
"""Load dataset from local directory"""


train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])

eval_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])


train_data = datasets.ImageFolder(data_dir, transform=train_transform)


train_loader = DataLoader(train_data, batch_size=16, shuffle=True)

print(f" Loaded {len(train_data)} training images")
print(f" Classes found: {train_data.classes}")

return train_loader, train_data, eval_transform
```

```
def train_model(model, train_loader, num_epochs=30):
    """Train the model"""



    class_counts = [len([x for x in os.listdir(os.path.join('dataset', cls)) if
x.endswith(('.jpg', '.png'))])
                    for cls in os.listdir('dataset') if os.path.isdir(os.path.join('dataset', cls))]
    total = sum(class_counts)
    class_weights = torch.tensor([total / c for c in class_counts],
dtype=torch.float).to(device)


    print(f"⚖️ Class weights: {class_weights.cpu().numpy()}")



    criterion = nn.CrossEntropyLoss(weight=class_weights)
    optimizer = optim.Adam(model.parameters(), lr=0.0005)
    scheduler = StepLR(optimizer, step_size=10, gamma=0.5)


    best_train_acc = 0


    for epoch in range(num_epochs):
        model.train()
        train_loss, correct, total = 0, 0, 0


        for images, labels in tqdm(train_loader, desc=f"Epoch
[{epoch+1}/{num_epochs}]"):
            images, labels = images.to(device), labels.to(device)
```

```
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            _, preds = torch.max(outputs, 1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        train_acc = 100 * correct / total
        avg_train_loss = train_loss / len(train_loader)

        scheduler.step()

        if train_acc > best_train_acc:
            best_train_acc = train_acc
            torch.save(model.state_dict(), "best_lung_cancer_model.pth")

        print(f"\n Epoch [{epoch+1}/{num_epochs}] Summary:")
        print(f"    Train Loss: {avg_train_loss:.4f} | Train Acc: {train_acc:.2f}%")
        print(f"    LR: {scheduler.get_last_lr()[0]:.6f}")
        print("-" * 55)

    print(f"\n Training Complete! Best Training Accuracy: {best_train_acc:.2f}%")
    return model
```

```
def test_model(model, test_dir, eval_transform, class_names):
    """Test the model on test images"""

    model.eval()



    test_images = []
    for root, dirs, files in os.walk(test_dir):
        for file in files:
            if file.endswith(('.jpg', '.png', '.jpeg')):
                test_images.append(os.path.join(root, file))

    print(f"\n Testing on {len(test_images)} images from {test_dir}")

    results = []

    for img_path in test_images:

        input_image = Image.open(img_path).convert("RGB")
        input_tensor = eval_transform(input_image).unsqueeze(0).to(device)



        with torch.no_grad():
            outputs = model(input_tensor)
            probs = torch.softmax(outputs, dim=1)
            pred_class = torch.argmax(probs, dim=1).item()
```

```
        result = {
            'image': os.path.basename(img_path),
            'predicted_class': class_names[pred_class],
            'confidence': probs[0][pred_class].item() * 100
        }
        results.append(result)

        print(f"📸 {result['image']}: {result['predicted_class']}
({result['confidence']:.2f}%)")

    return results



if __name__ == "__main__":
    print("=" * 60)
    print("🫁 Lung Cancer Classification Model - CSPDarkNet")
    print("=" * 60)


    if not os.path.exists('dataset'):
        print("❌ Error: 'dataset' folder not found!")
        exit(1)


    print("\n📂 Loading dataset...")
    train_loader, train_data, eval_transform = load_data('dataset')
```

```
num_classes = len(train_data.classes)
model = CSPDarkNet53(num_classes=num_classes).to(device)
print(f"\n CSPDarkNet53 model created for {num_classes} classes")



print("\n Starting training...")
model = train_model(model, train_loader, num_epochs=30)



if os.path.exists('test images'):
    print("\n" + "=" * 60)
    print(" Testing on images from 'test images' folder")
    print("=" * 60)
    results = test_model(model, 'test images', eval_transform, train_data.classes)



    with open('test_results.txt', 'w') as f:
        f.write("Lung Cancer Classification Results\n")
        f.write("=" * 60 + "\n\n")
        for r in results:
            f.write(f"{r['image']}: {r['predicted_class']} ({r['confidence']:.2f}%)\n")

    print("\n Results saved to 'test_results.txt'")
else:
    print("\n⚠ 'test images' folder not found. Skipping testing phase.")

print("\n" + "=" * 60)
print(" Process completed!")
print("=" * 60)
```

login.py:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Login - Lung Cancer Classification</title>

<style>

body{
   margin:0;
   font-family:Segoe UI, sans-serif;
   background:linear-gradient(135deg,#6a5acd,#7f7fff);
   display:flex;
   justify-content:center;
   align-items:center;
   height:100vh;
}

.card{
   width:380px;
   background:white;
   padding:35px;
   border-radius:16px;
   box-shadow:0 10px 30px rgba(0,0,0,0.2);
   text-align:center;
}

h1{
```

```
    margin-bottom:5px;
}

.subtitle{
    color:#666;
    margin-bottom:25px;
}

input{
    width:100%;
    padding:12px;
    margin:8px 0;
    border-radius:8px;
    border:1px solid #ddd;
    font-size:14px;
}

button{
    width:100%;
    padding:12px;
    border:none;
    border-radius:8px;
    background:#6a5acd;
    color:white;
    font-size:16px;
    margin-top:10px;
    cursor:pointer;
}
```

```
button:hover{
    background:#5746d4;
}

.error{
    color:#d9534f;
    margin-bottom:10px;
}

.link{
    margin-top:15px;
    font-size:14px;
}

a{
    color:#6a5acd;
    text-decoration:none;
}

</style>
</head>

<body>

<div class="card">

<h1>🫁 Login</h1>
<p class="subtitle">Access Lung Cancer Detection System</p>
```

```
{% if error %}
<p class="error">{{ error }}</p>
{% endif %}

<form method="POST">

<input type="text" name="username" placeholder="Username" required>

<input type="password" name="password" placeholder="Password" required>

<button type="submit">Login</button>

</form>

<div class="link">
Don't have an account?
<a href="/signup">Create one</a>
</div>

</div>

</body>
</html>
```

signup.py:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
```

```
<title>Signup - Lung Cancer Classification</title>

<style>

body{
    margin:0;
    font-family:Segoe UI, sans-serif;
    background:linear-gradient(135deg,#6a5acd,#7f7fff);
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}

.card{
    width:400px;
    background:white;
    padding:35px;
    border-radius:16px;
    box-shadow:0 10px 30px rgba(0,0,0,0.2);
    text-align:center;
}

h1{
    margin-bottom:5px;
}

.subtitle{
    color:#666;
```

```
    margin-bottom:25px;
}

input{
    width:100%;
    padding:12px;
    margin:8px 0;
    border-radius:8px;
    border:1px solid #ddd;
    font-size:14px;
}

button{
    width:100%;
    padding:12px;
    border:none;
    border-radius:8px;
    background:#6a5acd;
    color:white;
    font-size:16px;
    margin-top:10px;
    cursor:pointer;
}

button:hover{
    background:#5746d4;
}

.error{
```

```
    color:#d9534f;
    margin-bottom:10px;
}

.link{
    margin-top:15px;
    font-size:14px;
}

a{
    color:#6a5acd;
    text-decoration:none;
}

</style>
</head>

<body>

<div class="card">

<h1>🫁 Create Account</h1>
<p class="subtitle">Register to use the AI system</p>

{% if error %}
<p class="error">{{ error }}</p>
{% endif %}

<form method="POST">
```

<input type="text" name="username" placeholder="Username" required>

<input type="email" name="email" placeholder="Email" required>

<input type="password" name="password" placeholder="Password" required>

<input type="password" name="confirm_password" placeholder="Confirm Password" required>

<button type="submit">Create Account</button>

</form>

Already have an account?

Login

</body>

</html>

index.py:

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

```
<title>Lung Cancer Classification</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<style>
body{
    margin:0;
    font-family:Segoe UI, sans-serif;
    background:linear-gradient(135deg,#6a5acd,#7f7fff);
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}

.card{
    width:600px;
    background:white;
    padding:40px;
    border-radius:18px;
    text-align:center;
    box-shadow:0 10px 30px rgba(0,0,0,0.2);
}

h1{
    margin:0;
    font-size:28px;
}

.subtitle{
```

```
    color:#666;
    margin-bottom:30px;
}

.upload-box{
    border:2px dashed #9aa4ff;
    border-radius:12px;
    padding:40px;
    cursor:pointer;
    transition:0.2s;
}

.upload-box:hover{
    background:#f6f7ff;
}

.upload-icon{
    font-size:40px;
    margin-bottom:10px;
}

button{
    margin-top:20px;
    padding:12px 25px;
    border:none;
    border-radius:8px;
    background:#6a5acd;
    color:white;
    font-size:16px;
```

```
    cursor:pointer;
}

button:hover{
    background:#5746d4;
}

.result{
    margin-top:25px;
    font-size:18px;
}

.error{
    color:#d9534f;
}

.success{
    color:#28a745;
}

.preview{
    margin-top:20px;
    max-width:300px;
    border-radius:8px;
}
</style>
</head>

<body>
```

```
<div class="card">

<h1>🫁 Lung Cancer Classification</h1>
<p class="subtitle">Upload a lung CT scan image for AI-powered analysis</p>

<div class="upload-box" onclick="fileInput.click()">
<div class="upload-icon">📤 </div>
<p>Click to upload or drag and drop</p>
<p style="font-size:12px;color:#888;">PNG, JPG or JPEG (Max 16MB)</p>
</div>

<input type="file" id="fileInput" hidden accept="image/*">

<button onclick="analyzeImage()">Analyze Image</button>

<div id="preview"></div>

<div id="result" class="result"></div>

</div>

<script>

let selectedFile=null;

const fileInput=document.getElementById("fileInput");
const preview=document.getElementById("preview");
```

```
fileInput.addEventListener("change",function(){
    selectedFile=this.files[0];

    const reader=new FileReader();

    reader.onload=function(e){
        preview.innerHTML=`<img src="${e.target.result}" class="preview">`;
    }

    reader.readAsDataURL(selectedFile);
});

async function analyzeImage(){

    if(!selectedFile){
        alert("Please upload an image first");
        return;
    }

    const formData=new FormData();
    formData.append("file",selectedFile);

    const response=await fetch("/predict",{
        method:"POST",
        body:formData
    });

    const data=await response.json();
```

```
    const resultDiv=document.getElementById("result");

    if(data.error){
        resultDiv.innerHTML=`<p class="error">${data.error}</p>`;
        return;
    }

    if(data.predicted_class==="Unknown / Not a lung CT image"){
        resultDiv.innerHTML=`
            <p class="error">
            ⚠ Uploaded image does not appear to be a lung CT scan.
            Please upload a valid medical image.
            </p>
        `;
        return;
    }

    resultDiv.innerHTML=`
        <p class="success"><b>Prediction:</b> ${data.predicted_class}</p>
        <p><b>Confidence:</b> ${data.confidence.toFixed(2)}%</p>
    `;
    if(data.annotated_image){
        preview.innerHTML+=`<img src="${data.annotated_image}"
class="preview">`;
    }
}
</script>
</body>
</html>
```